standard chartered

AI Nucleus

Responsible AI Narrative Generation

December 2025

Tom Pfaff, Craig Turrell
GCFO

Abstract

We present a novel framework for quantifying and governing the temporal stability of Liquid Neural Networks (LNNs) within dynamic financial narrative graphs. By adapting Lyapunov exponents—a fundamental concept from dynamical systems theory—we derive a suite of interpretable metrics that serve as leading indicators of model robustness and behavioral predictability. These metrics, including the maximum Lyapunov exponent (`lambda_max`), unstable fraction, and derived stability entropy, are integrated directly into a production AI governance stack. This integration enables real-time stability dashboards, model risk reporting, and automated alerting through token-oriented metadata (TOON), OpenMetadata catalogs, and data lineage systems. Our framework transforms abstract stability analysis into actionable business intelligence, providing a mathematically rigorous foundation for model auditability, regulatory compliance, and operational risk management in high-stakes financial applications. An experimental implementation of a narrative Graph Neural Network (GNN) demonstrates effective monitoring and control of temporal dynamics, establishing a new paradigm for stability-aware AI deployment in non-stationary environments.

Introduction

The deployment of deep learning models in high-stakes, non-stationary environments such as finance and financial markets presents a unique challenge: models must not only be accurate but also **stable** over time. While Graph Neural Networks (GNNs) have shown remarkable success in modeling complex relational data, extending them to the temporal domain—particularly via recurrent or continuous-time architectures such as Liquid Neural Networks (LNNs)—introduces complex dynamical behaviors. These internal dynamics can range from stable and predictable to chaotic and highly sensitive to initial conditions. Traditional model validation metrics (e.g., loss, accuracy) are inherently static and fail to capture this crucial dimension of model behavior, creating a significant governance gap.

In financial applications, where narrative graphs model evolving relationships between entities, events, and risk factors, unstable model dynamics pose a tangible business risk. A model with chaotic internal-state propagation may produce plausible outputs on historical data but become wildly unpredictable during market regime shifts or edge-case scenarios. This instability remains invisible to standard monitoring tools, potentially leading to undetected model degradation and erroneous decision-making.

To address this, we turn to dynamical systems theory, specifically the concept of Lyapunov exponents. A Lyapunov exponent quantifies the average exponential rate of divergence or convergence of nearby trajectories in a dynamical system. This provides a direct, quantitative measure of stability or chaos. However, applying this classical theory to modern, learned neural network components within a production AI stack presents significant technical and practical hurdles. The metric must be not only computationally feasible but also interpretable, integrable into existing ML pipelines, and actionable for model risk management.

In this paper, we introduce a framework that bridges this gap. We make the following core contributions:

1. **A practical method for estimating Lyapunov stability in Liquid Neural Networks.** We adapt the classical Benettin algorithm to operate efficiently within the forward pass of an LNN-based narrative GNN, providing numerical stability metrics (`lambda_max`, `entropy`) for individual nodes and aggregated storylines.
2. **A novel integration of stability metrics into the AI governance lifecycle.** We define stability as a first-class, queryable property within the model's metadata. By embedding Lyapunov metrics into token-oriented objects (TOON) and OpenMetadata-style catalogs, we enable stability-aware model discovery, lineage tracing, and risk reporting.
3. **An operational stability dashboard and alerting protocol.** We translate continuous stability metrics into a discrete, traffic-light system (GREEN/AMBER/RED) with business-interpretable thresholds, enabling real-time monitoring and governance workflows familiar to risk officers.
4. **A differentiable stability regularizer.** We provide a method to incorporate stability constraints directly into the training objective, allowing models to be explicitly optimized for robust temporal dynamics without sacrificing task performance.

This work establishes a new paradigm for **stability-aware AI governance**. It moves beyond post hoc analysis, embedding dynamic safety directly into the development, deployment, and monitoring lifecycle. We demonstrate the framework's implementation within a financial narrative modeling system, showing how it turns the abstract mathematical concept of Lyapunov stability into a critical tool for building more reliable, transparent, and trustworthy temporal models. The remainder of this paper is organized as follows: Section 2 reviews related work. Section 3 provides the necessary mathematical background—Section 4 details our implementation architecture. Section 5 describes the integration with metadata and governance stacks. Section 6 presents guidelines for interpretation and use. Finally, Section 7 discusses broader implications and future work.

Related Work

Our work sits at the intersection of temporal graph neural networks, dynamical systems analysis of machine learning models, and AI governance in financial technology. We review the relevant literature in these domains.

Temporal and Dynamic Graph Neural Networks

Graph Neural Networks (GNNs) have evolved to model temporal dynamics, leading to Dynamic Graph Neural Networks (DGNNs). Key approaches include discrete-time methods that operate on graph snapshots and continuous-time methods that model events as point processes on graphs. Liquid Neural Networks (LNNs), inspired by differential equations and continuous-time RNNs, represent a significant advance for modeling temporal dependencies with stable, causal dynamics. Their application in GNNs for financial narrative modeling—where nodes and edges have continuously evolving states—serves as the substrate of our study. Unlike prior work focusing primarily on predictive accuracy, we address the *dynamical stability* of these temporal propagations.

Stability Analysis in Machine Learning

The stability and robustness of neural networks are traditionally studied in the context of adversarial examples and input perturbation sensitivity. However, the *internal* temporal stability of recurrent or continuous-time models is a distinct concern rooted in dynamical systems theory. Lyapunov exponents are the standard measure for quantifying chaos and stability in dynamical systems. Their application to traditional RNNs has been explored in theoretical machine learning, but practical integration into the training and governance of production GNN systems remains unaddressed. Our work bridges this gap by providing efficient estimators and a governance framework tailored for complex narrative graphs.

AI Governance, Risk Management, and Model Monitoring

The rise of operational AI has necessitated robust governance frameworks. Initiatives like Model Cards, FactSheets, and the integration of metadata catalogs (e.g., OpenMetadata) aim to improve transparency. In finance, model risk management (MRM) frameworks require demonstrable model validation and monitoring. Current monitoring focuses on performance drift and shifts in data distribution. Our contribution introduces *dynamical stability* as a new, critical dimension of model health, extending monitoring beyond statistical metrics to the intrinsic behavior of the model's temporal engine. Embedding stability metrics in lineage and metadata systems provides the audit trail required by modern governance standards.

Financial Narrative and Knowledge Graph Modeling

The use of knowledge graphs and narrative structures in finance is established for risk assessment and event detection. Representing financial events and relationships as dynamic graphs allows for causal reasoning and scenario analysis. However, the stability of the underlying dynamical models used for narrative propagation has not been formally assessed. Our work directly addresses this oversight by providing tools that ensure narrative forecasting and simulation are built on stable, predictable dynamics.

Mathematical Background

This section establishes the formal dynamical systems framework underpinning our stability metrics. We focus on the Lyapunov exponent as a quantitative measure of trajectory divergence in nonlinear systems and detail its adaptation to discrete-time stepping in a Liquid Neural Network.

Dynamical Systems Formalism

Consider a dynamical system defined by a state vector $\mathbf{h} \in \mathbb{R}^d$and an evolution rule. In our context, this system is the hidden state of a node or storyline within the LNN temporal core. The evolution can be expressed as a discrete-time map:

$\mathbf{h}_{t+1} = \mathbf{F}(\mathbf{h}_t, \mathbf{x}_t, \Delta t_t),$

Where $\mathbf{F}$ is the learned LNN state transition function,$\mathbf{x}_t$ iis an input feature vector (e.g., aggregated graph messages), andΔt_t is the time step increment. This formulation treats the LNN as an autonomous dynamical system when conditioned on a fixed input sequence.

Lyapunov Exponents: Definition and Interpretation

The Lyapunov exponent (λ) characterizes the average exponential rate of separation of infinitesimally close trajectories. For a d-dimensional system, there exists a spectrum of d exponents $\lambda_1 \geq \lambda_2 \geq \ldots \geq \lambda_d$. The dominant, or maximum Lyapunov exponent (MLE), $\lambda_{\max}$, is defined for two initially close states$\mathbf{h}_0$ and $\mathbf{h}_0 + \boldsymbol{\delta}_0$ with $\|\boldsymbol{\delta}_0\| \to 0$ as:

$$\lambda_{\max} = \lim_{T\to\infty} \frac{1}{T} \lim_{\|\boldsymbol{\delta}_0\|\to 0} \ln \frac{\|\boldsymbol{\delta}_T\|}{\|\boldsymbol{\delta}_0\|},$$

Where $\boldsymbol{\delta}_T = \mathbf{h}_T - \mathbf{h}'_T$ is the separation after time T.

The MLE's sign provides the fundamental stability classification:
- $\lambda_{\max} < 0$: Trajectories converge exponentially (stable, dissipative system).
- $\lambda_{\max} = 0$: Trajectories maintain constant separation on average (marginally stable, periodic dynamics).
- $\lambda_{\max} > 0$: Trajectories diverge exponentially (chaotic, sensitive to initial conditions).

Numerical Estimation via the Benettin Algorithm

For practical computation in learned systems, we employ a finite-time, finite-perturbation variant of the standard Benettin (or Jacobian-based) algorithm. Given an initial state $\mathbf{h}_0$ and a fixed input sequence, the procedure is:

Initialization: Generate a random perturbation vector $\boldsymbol{\xi}_0$ with unit norm $\|\boldsymbol{\xi}_0\| = 1$. Set a small amplitude ϵ (e.g., 10^{-6})) and define the perturbed state \(\mathbf{h}'_0 = \mathbf{h}_0 + \epsilon \boldsymbol{\xi}_0 \).

Evolution and Rescaling: For each step \(k = 0, 1, ..., K-1 \):
- Evolve both states: $\mathbf{h}_{k+1} = \mathbf{F}(\mathbf{h}_k, \mathbf{x}_k, \Delta t_k)$, $(\mathbf{h}'_{k+1} = \mathbf{F}(\mathbf{h}'_k, \mathbf{x}_k), \Delta t_k)$
- Compute the separation vector: $\boldsymbol{\delta}_{k+1} = \mathbf{h}'_{k+1} - \mathbf{h}_{k+1})$.
- Record its magnitude: $d_{k+1} = \|\boldsymbol{\delta}_{k+1}\|$
- Renormalize the perturbation for the next step: $\boldsymbol{\xi}_{k+1} = \boldsymbol{\delta}_{k+1}/d_{k+1}$
- Reset the perturbed state: $\mathbf{h}'_{k+1} = \mathbf{h}_{k+1} + \epsilon\boldsymbol{\xi}_{k+1}$

Estimation: The finite-time MLE $\hat{\lambda}_{\max}$ is calculated as:

$$\hat{\lambda}_{\max} = \frac{1}{T} \sum_{k=0}^{K-1} \ln \frac{d_{k+1}}{\epsilon}, \quad \text{where } T = \sum_{k=0}^{K-1} \Delta t_k \,.$$

This algorithm tracks the growth of a perturbation aligned with the most unstable direction in tangent space.

Stability Metrics for Narrative Governance

From the core estimator, we derive three key operational metrics:
- **Maximum Lyapunov Exponent (λ_max):***The primary stability indicator, computed as above.

- **Stability Entropy:** A non-negative measure of chaotic complexity, defined as $H = \max(\hat{\lambda}_{\max}, 0)$. This metric is zero for stable systems and positive for unstable ones.
- **Unstable Fraction:** For a set of \(N \) narrative nodes or trajectories, the fraction for which $\hat{\lambda}_{\max} > 0$. This provides a system-level view of the prevalence of instability.

These metrics underpin the traffic-light classification system (GREEN/AMBER/RED) used in operational dashboards. The classification is based on the integrated growth factor of a perturbation over a predefined business-relevant horizon $\tau G = \exp(\hat{\lambda}_{\max} \cdot \tau)$. Thresholds are set as:
GREEN: $G \leq 2$ (perturbation at most doubles).
AMBER: $2 < G \leq 10$ (perturbation grows up to one order of magnitude).
RED: $G > 10$(perturbation grows more than one order of magnitude).

This mathematical foundation provides a rigorous, quantifiable basis for assessing and governing the temporal stability of narrative models, thereby transitioning dynamical systems theory into a practical, responsible AI risk-management tool.

Implementation

This section details the software architecture and core components that operationalize the Lyapunov stability framework within the narrative Graph Neural Network (GNN) system. The design is modular, ensuring that stability metrics are an opt-in enhancement that does not disrupt existing model functionality.

System Overview

The architecture extends the base `MultiPurposeNarrativeGNN` with an `EnhancedNarrativeGNN` that incorporates a `LiquidStateUpdater` to capture temporal dynamics. The Lyapunov metrics system is implemented across three layers:

- **Core Utilities:** Low-level functions for the Benettin algorithm and metric computation.
- **Model-Level APIs:** Methods integrated into the GNN to compute stability for nodes and storylines.
- **Training Integration:** Hooks for logging and regularization within the training loop.

Core Utilities: The `dynamical_metrics` Module

This foundational module provides numerical routines for stability analysis that are independent of the specific model architecture.

**LyapunovMetrics` Dataclass:** A structured container for results, including `lambda_max` (estimated MLE), `entropy` (derived stability entropy), `num_steps`, and `std_error` (standard error from multiple runs).
`estimate_max_lyapunov` Function: A generic implementation of the finite-time Benettin algorithm. Its signature is:

```
def estimate_max_lyapunov(step_fn, initial_state, input_sequence, dt_sequence,
epsilon=1e-6):
```

It takes a callable `step_fn` that defines the state transition $\mathbf{F}$, making it reusable for any differentiable dynamical system.

- **`analyze_lnn_state_updater` Function:** A specialized adapter that wraps the `LiquidStateUpdater` module into the required `step_fn` interface, handling the batching and autograd context for evaluation.
- **`lyapunov_regularization_term` Function:** Returns a differentiable loss term `max(estimated_lambda_max - target, 0)`. This enables gradient-based minimization of training instability.

Model-Level APIs: The `EnhancedNarrativeGNN`

This class extends the narrative GNN, providing methods that bridge the core utilities with the model's graph-structured data.

`measure_node_lyapunov` Method: Computes the Lyapunov spectrum for a specific node's trajectory over a given message and time-step sequence. It internally reconstructs the node's state evolution using the LNN updater and calls the core estimation utilities.
`measure_storyline_stability` Method: Aggregates stability across all nodes belonging to a defined storyline (e.g., "liquidity stress"). It returns a summary dictionary:

```
{
 'lambda_max_mean': np.mean(node_lyapunovs),
 'unstable_fraction': np.mean([l > 0 for l in node_lyapunovs]),
 'entropy_mean': np.mean([max(l, 0) for l in node_lyapunovs]),
 'stability_status': 'GREEN'  # Derived from thresholds
}
```

`lyapunov_regularization_for_storyline` Method: Samples nodes from a specified storyline, computes their differentiable Lyapunov estimates, and returns a scalar penalty term suitable for addition to the primary training loss. This is the key method for stability-aware training.

Training Integration

The framework is integrated into the model lifecycle via a modified training script (`train_financial_terms_enhanced.py`).

Periodic Diagnostics: A `--log_lyap_every N` flag triggers the calculation and logging of storyline stability metrics on a validation set every N epochs. This provides a time series of stability during training, independent of any regularization.
Stability-Aware Loss: The training loss $\mathscr{L}_{\text{total}}$ is extended when the `--lyap_reg_weight` α s > 0:

$$\mathscr{L}_{\text{total}} = \mathscr{L}_{\text{task}} + \alpha \cdot \mathscr{L}_{\text{lyap}}$$

where $\mathscr{L}_{\text{lyap}}$ is the output of `lyapunov_regularization_for_storyline`.

Hyperparameters: Key controls include:
*`lyap_lambda_target`: The stability target (typically 0 or slightly negative).
*`lyap_horizon`: The physical time horizon \(\tau \) over which perturbation growth is evaluated for the traffic-light status.

Operational Modes

The architecture supports two primary usage modes, configurable via hyperparameters:
- **Diagnostic Mode (`lyap_reg_weight = 0`):** Passive monitoring. Stability metrics are computed and logged for model validation and risk reporting without affecting the training dynamics.
- **Regularized Training Mode (`lyap_reg_weight > 0`):** Active stabilization. The model is explicitly penalized for exhibiting unstable dynamics, guiding it towards a more robust region of its parameter space.

This layered architecture ensures that Lyapunov stability metrics are not a post-hoc analysis but a first-class, computable property of the narrative model, deeply integrated into both its development and operational lifecycle.

AI Governance and Metadata

A core contribution of this work is the operationalization of Lyapunov stability metrics within a production AI governance ecosystem. The metrics are not merely computational outputs but are engineered as first-class, queryable metadata that flow through the entire model lifecycle. This section details the integration points with the Token-Oriented Object Notation (TOON), OpenMetadata standard, telemetry systems, and data lineage graphs.

Token-Oriented Representation (TOON)

Each narrative storyline and model component is represented as a TOON object—a structured, tokenized data entity that encapsulates state, metadata, and capabilities. Lyapunov stability metrics are embedded as first-class properties within these tokens.

Schema Extension: The TOON schema for a `NarrativeStoryline` or `ModelInstance` is extended to include a `stability` object:

```
 {
"type": "NarrativeStoryline",
"id": "nsl:liquidity_stress:2024Q1",
"attributes": { ... },
"stability": {
  "lambda_max_mean": -0.05,
  "unstable_fraction": 0.10,
  "entropy_mean": 0.00,
  "status": "GREEN",
  "horizon_days": 30,
  "last_updated": "2024-03-15T14:30:00Z"
}
 }
```

Immutability and Versioning: Each stability snapshot is appended immutably to the token's history, creating an auditable record of model behavior over time. This allows for comparisons across model versions and retraining cycles.

OpenMetadata Catalog Integration

The framework aligns with OpenMetadata's standardized approach to data and model cataloging. Stability metrics are exposed as custom **model metadata** attributes, making them discoverable and filterable through the central catalog.

Asset Annotation: The narrative model assets (`MLModel` type in OpenMetadata) are annotated with a `lyapunovStability` field in their `extension` property.

Query and Filter Capability: Data stewards and risk officers can query the catalog using standard APIs or UI filters to, for example:

*"List all storylines with `stability_status = 'RED'`."

*"Find models where `lambda_max_mean` increased by more than 0.1 in the last week."

Impact Analysis: By linking model assets to downstream dashboards, reports, and decision systems, the catalog enables impact analysis. An unstable model can be traced to all dependent consumer assets for prioritized review.

Telemetry and Production Monitoring

Stability metrics are emitted as structured telemetry events during both training and inference and integrate seamlessly with observability stacks.

Event Schema: Telemetry events are tagged with the critical identifiers (`storyline_id`, `model_version`, `experiment_id`) and include the `LyapunovMetrics` payload.

Dashboarding: The `stability_status` and underlying numeric metrics are visualized in real-time operational dashboards alongside traditional KPIs like latency, throughput, and prediction accuracy. A "Narrative Stability" dashboard panel provides an at-a-glance view of system health.

Alerting: Alerting rules can be configured on the telemetry stream. For example:

*Trigger a warning if any storyline's status shifts from GREEN to AMBER.

*Trigger a critical alert if a key storyline's `lambda_max_mean` becomes positive (`> 0`) for three consecutive measurement periods.

Lineage and Provenance Integration

Data lineage is critical for understanding dependencies and diagnosing issues. The framework injects stability metadata into the lineage graph, creating **stability-aware lineage.**

Node-Level Annotation: Each node in the lineage graph representing a model run or storyline computation is annotated with its computed stability metrics.

Visual Highlighting: In lineage visualization tools, nodes can be color-coded (green/amber/red) based on their `stability_status`, instantly highlighting potential sources of instability in a complex pipeline.

Provenance for Decisions: When a downstream report or trading signal is generated, its full lineage—including the stability state of each contributing narrative model at execution time—can be retrieved. This provides a complete provenance record for audit and explainability, answering not just *what* data was used, but also *how stable* the models that processed it were.

End-to-End Governance Workflow

The integrated stack enables a closed-loop governance workflow:

Detection: The `EnhancedNarrativeGNN` computes stability metrics during a scheduled evaluation job.

Publication: Metrics are published as TOON token updates and emitted as telemetry.

Cataloging & Discovery: The OpenMetadata catalog is updated, making the new stability state searchable.

Monitoring & Alerting: The telemetry stream is monitored. A status change to AMBER triggers an alert to the model owner.

Investigation: The owner uses the lineage graph to see the unstable component and its dependencies. They query the catalog for historical stability trends.

Action: The owner may trigger model retuning with a higher `lyap_reg_weight`, quarantine the unstable storyline, or annotate the model card with a risk warning for downstream users.

Verification: After intervention, subsequent stability metrics are tracked to verify a return to GREEN status.

Usage and Interpretation Guidelines

This section provides practical guidance for implementing and interpreting the Lyapunov stability framework within financial narrative modeling. Proper interpretation is essential for translating theoretical metrics into actionable insights for model development, validation, and risk management.

Operational Interpretation of Core Metrics

The framework produces a suite of complementary metrics, each offering a distinct perspective on model stability.

The **Maximum Lyapunov Exponent (λ_max)** serves as the primary indicator. A negative value indicates inherent stability, in which small perturbations decay exponentially. A positive value suggests intrinsic instability and chaotic divergence. Values approximating zero denote marginal stability—a regime of high sensitivity without explosive growth. In practice, a slightly negative λ_max (typically between -0.1 and 0.0) is often desirable in financial narrative models, balancing responsiveness with necessary damping. The **Unstable Fraction** quantifies the prevalence of instability within a storyline by reporting the proportion of constituent nodes with a positive λ_max. A low value (below 0.2) coupled with a negative mean λ_max suggests overall stability with isolated exceptions. A high value (above 0.5) indicates widespread instability warranting immediate investigation into data quality, model architecture, or training convergence.

Stability Entropy, defined as `max(λ_max, 0)`, provides a non-negative measure of chaotic complexity. Zero entropy denotes the absence of exponential divergence. Positive entropy directly quantifies the rate of information loss and effectively limits the model's trustworthy prediction horizon.

The synthesized **Stability Status (GREEN/AMBER/RED)** provides an immediate operational indicator. This classification is derived from the estimated perturbation growth factor, $G = \exp(\lambda_{\text{max}} \cdot \tau)$, over a defined business-relevant horizonτ (e.g., 30 days). A GREEN status corresponds to a growth factor $G \leq 2$, meaning a perturbation at most doubles. An AMBER status indicates growth between 2 and 10 times (one order of magnitude). A RED status signifies growth exceeding one order of magnitude $G > 10$.

Guidelines for Model Development and Tuning

During model development and training, stability metrics should inform iterative refinement. In **diagnostic mode**, stability metrics provide a crucial validation signal beyond traditional loss. A sharp increase in λ_max or unstable fraction during training can indicate overfitting to noise, inappropriate architecture choices, or poorly conditioned gradients. Training should be monitored for convergence across performance and stability metrics. When employing **regularized training** (with a positive `lyap_reg_weight`), the target λ_max should be set conservatively, typically between -0.05 and 0.0. The regularization weight must be carefully balanced; a value that is too high can over-constrain the model, leading to excessively damped dynamics and underfitting, while a value that is too low may be ineffective. A recommended starting point is a weight of 1e-3, adjusted based on the stability-performance trade-off observed on validation data. Hyperparameter sweeps should include stability as a key optimization criterion. Configurations that yield marginally better accuracy but significantly worse stability (e.g., a shift from AMBER to RED) should be rejected in favor of more robust alternatives.

Production Monitoring and Alerting Protocol

In production, stability metrics must be integrated into standard model monitoring dashboards and alerting pipelines. A baseline stability profile should be established for each monitored storyline using validation data from a stable historical period. This baseline serves as a reference for detecting deviations. Alerts should be configured hierarchically: a **warning** triggered by a status transition from GREEN to AMBER, and a **critical** alert triggered by a transition to RED or a sustained positive λ_max over multiple evaluation periods.

Investigation of an alert should follow a structured triage:

Data Interrogation: Check for regime shifts, outliers, or missing data in the input features feeding the unstable storyline.

Model Consistency: Verify that the model version and configuration have not changed unexpectedly.

Lineage Analysis: Use the stability-annotated lineage graph to determine if instability originates from a particular data source or upstream model component.

Scenario Testing: Run controlled stress tests to see if specific, plausible edge cases trigger instability.

Reporting and Governance Integration

For model risk governance, stability metrics provide auditable evidence of model robustness. Model validation reports should include a dedicated stability assessment section that documents the λ_max, unstable fraction, and status for all critical storylines at initial sign-off and during periodic revalidation. The rationale for the chosen stability horizon (τ) and the classification thresholds must be explicitly documented and justified based on the business context. For instance, a liquidity risk model may warrant a shorter horizon and stricter thresholds than a long-term strategic narrative model. Finally, stability status should be incorporated into model deployment gates. A model with any storyline in RED status should not be promoted to production without explicit risk acceptance from the model governance committee. An AMBER status may require additional monitoring controls and a defined remediation plan.

Conclusion and Future Work

We have presented a comprehensive framework for quantifying, monitoring, and governing the temporal stability of narrative-driven AI models in financial applications. By adapting the mathematical formalism of Lyapunov exponents to the context of Liquid Neural Networks within dynamic graph architectures, we have established a direct, numerical measure of model robustness that complements traditional performance metrics. Our implementation demonstrates that stability metrics can be efficiently computed, seamlessly integrated into

production AI stacks via token-oriented metadata and OpenMetadata catalogs, and operationalized through intuitive traffic-light dashboards and alerting systems.

The primary contribution of this work is to transform an abstract dynamical systems concept into a foundational element of modern AI governance. We have shown that stability is not merely a theoretical concern but a practical, measurable attribute that directly impacts model reliability in non-stationary environments. By embedding Lyapunov metrics into the model lifecycle—from differentiable regularization during training to lineage-aware monitoring in production—we provide a closed-loop system for stability assurance. This addresses a critical gap in the validation and risk management of complex temporal models, particularly in regulated sectors like finance, where predictability and auditability are paramount.

Looking ahead, several research and engineering directions naturally follow from this foundation. First, extending the analysis from the maximum exponent to the full Lyapunov spectrum would provide a more detailed characterization of the attractor geometry and intrinsic dimensionality of the learned dynamics. Second, developing methods to attribute instability to specific subgraph structures, edge types, or input features would enhance debuggability and inform architectural improvements. Third, integrating stability constraints directly into automated hyperparameter optimization and neural architecture search pipelines could yield inherently robust model designs. Finally, exploring the application of this framework beyond financial narratives to other domains with critical temporal dynamics—such as autonomous system planning, or financial outcome forecasting—represents a promising avenue for broader impact.

Pilot Readiness Action Plan

Theoretical foundations & metric design – **GREEN**

- Clear exposition of Lyapunov exponents, Benettin algorithm, and finite-time estimation.
- Appropriate adaptation to discrete-time LNN dynamics and narrative nodes/storylines.
- Derived metrics (λ_max, unstable fraction, "stability entropy") are well-motivated and interpretable.
- Traffic-light thresholds (GREEN/AMBER/RED) tied to perturbation growth over a business-relevant horizon τ serve as a strong bridge from mathematical to business semantics.

Software architecture & APIs – **AMBER**

Architecture is coherent and modular:

- dynamical_metrics core utilities with estimate_max_lyapunov, adapters, and a LyapunovMetrics dataclass.
- EnhancedNarrativeGNN with node/storyline-level stability methods.
- Training integration via train_financial_terms_enhanced.py and hyperparameters (lyap_reg_weight,
- lyap_lambda_target, lyap_horizon).
- Good separation of concerns (core utilities vs. model-level APIs vs. training hooks).

Gaps:

- No information on unit tests, numerical validation, or edge-case handling.
- No stated performance characteristics (cost vs. baseline; overhead on large graphs).
- No discussion of deterministic behavior, seeding, or reproducibility, despite stochastic procedures
- (random perturbations, sampling).

Training-time integration & regularization – **AMBER**

Design supports two modes:

- Diagnostic mode (lyap_reg_weight = 0) – passive monitoring.
- Regularized mode (lyap_reg_weight > 0) – active stabilization.
- A simple, interpretable regularizer (max(lambda_est - target, 0)) is conceptually sound.

Guidance is given on:

- Reasonable target ranges for λ_max (slightly negative around [-0.1, 0]).
- Initial regularization weights and the trade-off with task performance.

Gaps:

No empirical evidence of how regularization affects:

- Predictive accuracy.
- Training stability.
- Over- or under-damping in real financial narratives.
- No documented strategy for hyperparameter tuning with stability as a primary criterion (though suggested
- qualitatively).

Governance, metadata, and lineage integration – **AMBER**

- Strong conceptual alignment with modern governance:
- Stability as first-class metadata in TOON tokens and OpenMetadata catalogs.
- Immutable, versioned stability snapshots for auditability.
- Stability-aware lineage graphs and provenance for downstream decisions.
- Proposed queries and workflows are realistic (e.g., "list all RED storylines", impact analysis).

Gaps:

- No details on mapping to actual MRM/Model Governance policies (role approvals, sign-offs, RAS limits).
- No discussion of integration with existing model card / FactSheet processes beyond a conceptual mention.
- No explicit treatment of privacy/security hardening around telemetry and metadata.

Monitoring, dashboards, and alerting – **AMBER**

- Telemetry schema and typical alerts are well-defined (storyline IDs, model version, experiment IDs;
- warnings on GREEN→AMBER, critical on sustained RED/ λ>0).
- Real-time dashboards with stability panels integrated with existing KPIs are described.
- Lineage-based visual highlighting (green/amber/red nodes) is a strong design idea.

Gaps:

- No mention of:
- Rate-limiting or deduplicating noisy alerts.
- SLOs/SLAs around stability computation.
- Fallback behavior when metrics cannot be computed (time-outs, partial failures).
-